VERIQTA

First 90 Days as an SRE

20 Production Challenges, Real Projects, and a Reliability Readiness Review



VERIQTA

SLOs & Reliability | **Incident Response** | **Deployment Safety** | **Disaster Recovery**

Days 1 – 30
Discover & Stabilize

Days 31 – 60
Improve & Automate

Days 61 – 90
Prove, Scale & Own

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

Your First Day on Call: The System Is Green, but Customers Are Complaining

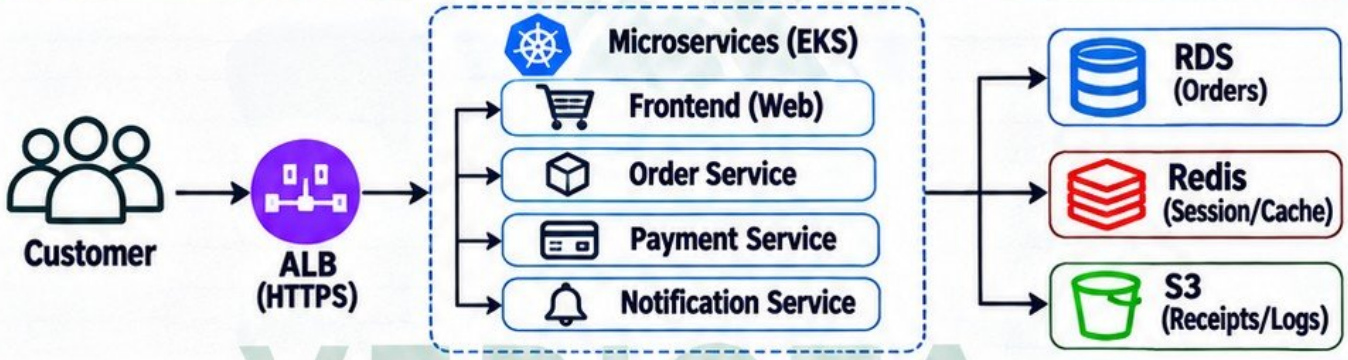
The Situation

Infrastructure dashboards show healthy CPU, memory, and pods, but checkout requests are failing and customers cannot complete payments.

What You Will Do

- Investigate the mismatch between system health and user experience.
- Reconstruct the customer journey from browse to successful checkout.
- Identify monitoring blind spots that hide business impact.

Project: Build a Production Service Map and Customer-Impact Dashboard



Deliverable

- Service ownership map (who owns what and where it runs).
- Critical dependencies (internal and external).
- First reliability risk register (risks, impact, and next actions).

Key Outcomes

- End-to-end view of the checkout journey.
- Visibility into user impact, not just infrastructure.
- A prioritized list of reliability and monitoring gaps.

VERIQTA

The Availability Number That Hides Your Most Valuable Customers

The Situation

An API can meet its availability target (e.g., 99.9%) while payment completion, premium customers, or a specific region suffer much higher failure rates.

What You Will Do

- Look beyond global averages to customer and journey-level reliability.
- Measure real user outcomes (checkout, payment, and order completion).
- Compare global vs. segmented reliability (by customer tier, region, and channel).

Project: Build Journey-Based SLIs



Segmented Analysis

- Premium Customers
- By Region (e.g., EU, US, AFR)
- By Channel (Web, Mobile, API)

Example: Global vs. Segmented Reliability

Journey / SLI	Global (Avg)	Premium Users	EU Region	Mobile App
Checkout Success	99.95%	99.60%	99.40%	99.72%
Payment Confirmation	99.93%	99.20%	99.10%	99.50%
Order Completion	99.90%	99.30%	99.15%	99.45%

SLO Specification (Example)

- Checkout Success SLO: ≥ 99.95% / 30 days
- Payment Confirmation SLO: ≥ 99.90%
- Order Completion SLO: ≥ 99.90%
- Measurement: Rolling 30 days, by global and segmented views

Error-Budget Calculation

SLO: 99.90% (30 days)

Total Requests: 1,000,000

Allowed Errors (Budget): 0.10%

= 1,000,000 × 0.001 = 1,000 requests

Deliverable

- A full SLO specification with justified targets.
- Journey-based SLIs (checkout, payment, order).
- A global vs. segmented reliability comparison.
- An error-budget calculation with action thresholds.

The Alert That Woke Six Engineers and Helped Nobody

The Situation

Noisy CPU alerts, duplicated pages, and missed customer-impacting failures are waking engineers, yet real business issues still slip through.

What You Will Do

- Analyze current alert noise and identify duplicated or low-value alerts.
- Separate actionable symptoms from diagnostic signals.
- Use error-budget burn-rate alerts to detect real customer impact early.

Project: Replace Noisy Alerts with Error-Budget Burn-Rate Alerts



Alert Policy (Example)

Condition	Window	Threshold	Action
Fast burn (major outage)	5 min	> 14×	Page (P1)
Sustained burn (degrading)	1 hour	> 2×	Page (P2)
Low noise (diagnostic)	15 min	> 1×	Ticket (P3)

Alert Routing Matrix

Severity	Who Gets It	Channel	When
P1 – Major	On-Call SRE + Lead	Pager / X / Chat	24/7
P2 – Degraded	SRE On-Call	Slack / Telegram	Business hrs
P3 – Inform	Service Owner	Ticket (GitHub)	Next cycle

Escalation Rules

- Noack within 5 minutes → escalate to Lead.
- No resolution within 30 minutes → escalate to Engineering Manager.
- Customer impact (SLO breach) → trigger incident (P1) immediately.

Deliverable

- A production-ready alert policy (SLO + burn-rate).
- A routing matrix (who, where, and when).
- Escalation rules with clear time thresholds.
- Measured result: reduced false pages (e.g., 70% → 10%).

The Dependency Nobody Knew Could Take Down Production

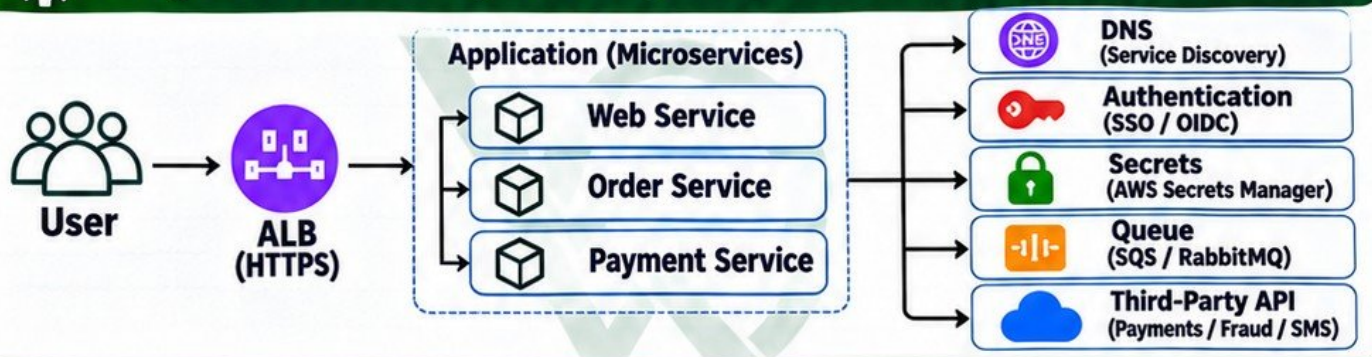
The Situation

Trace hidden dependencies involving DNS, authentication, secrets, queues, and third-party APIs. A dependency that looks noncritical can become a shared failure domain and take down production.

What You Will Do

- Map and verify all direct and indirect dependencies.
- Analyze how a failure propagates across services.
- Identify shared failure domains (e.g., DNS, Auth, Secrets, Queues, APIs).
- Pinpoint high-risk dependencies and containment options.

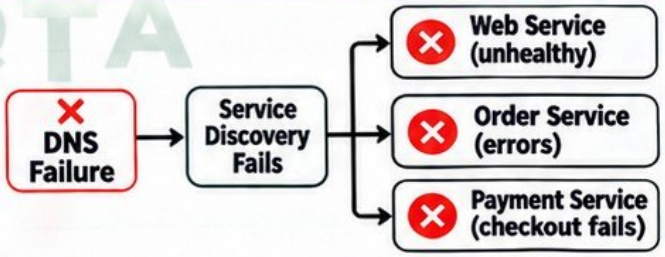
Project: Build a Dependency Graph and Test Failure Scenarios



Dependency Criticality Matrix

Dependency	Criticality	Impact if it Fails	Action
DNS	High	Service discovery fails (multiple services)	Multi-AZ + failover
Authentication	High	Users cannot log in	HA + cache tokens
Secrets Store	Medium	Services cannot get credentials (partial outage)	Retry + local cache
Message Queue	Medium	Orders delayed (backlog builds)	DLQ + monitoring
Third-Party API	Low	Noncore feature fails (e.g., notifications)	Timeout + circuit breaker

Failure Propagation (Example)



Containment Recommendations

- Reduce blast radius (separate critical and noncritical paths).
- Use redundancies (multi-AZ DNS, Auth, and Secrets).
- Apply timeouts, retries, and circuit breakers.
- Add dependency health checks and fallback behavior.

Deliverable

- A dependency criticality matrix (with impact and owner).
- A failure propagation diagram (how the outage spreads).
- A containment and mitigation report (with next actions).

VERIQTA

The Kubernetes Cluster Has Capacity, but the Application Cannot Scale

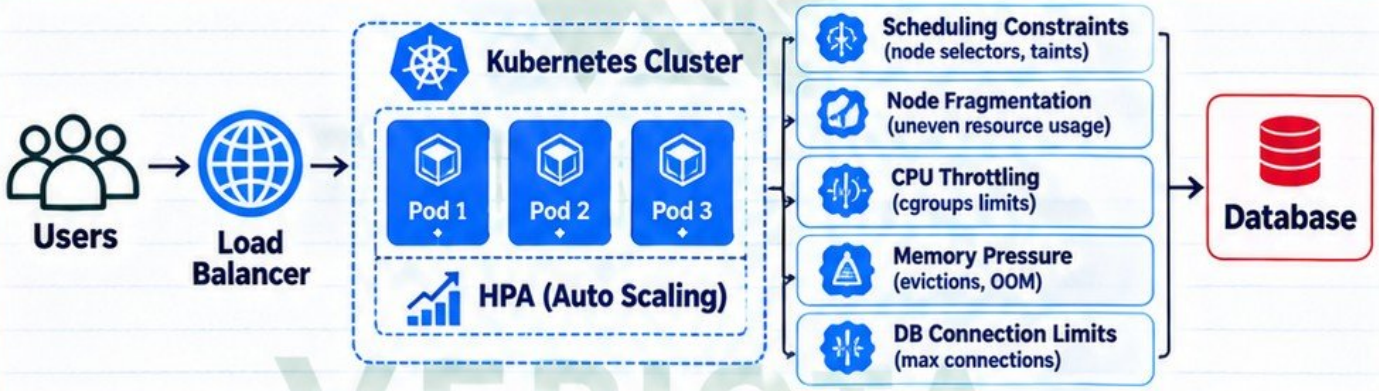
The Situation

The cluster shows available CPU and memory, but adding more replicas does not improve throughput. The application remains slow, and users still experience delays.

What You Will Do

- Investigate scheduling constraints, node fragmentation, CPU throttling, memory pressure, HPA behavior, and database connection limits.
- Reproduce a workload where adding replicas fails to improve throughput.
- Identify the actual limiting resource and redesign the scaling policy.

Project: Reproduce the Scaling Failure and Redesign the Policy



Key Metrics to Analyze

- CPU usage and throttling.
- Memory usage and pressure.
- Pod scheduling and events.
- HPA desired vs. actual replicas.
- Database connections and latency.

Before vs. After Load Test

Metric	Before	After
Throughput (rps)	320	1,280
Average Latency (ms)	820	240
CPU Usage (%)	45	62
Memory Usage (%)	52	68
DB Connections	180	400

Deliverable

- Bottleneck analysis (with evidence).
- Before-and-after load-test results.
- Capacity thresholds (for each resource).
- Redesigned scaling policy (HPA + resource limits).

The Retry Policy That Turned a Small Failure Into a Regional Incident

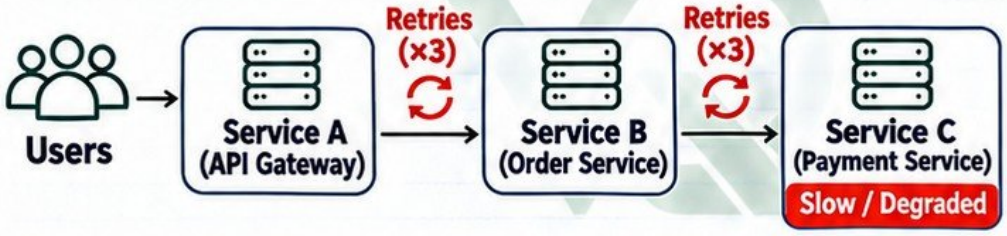
The Situation

A minor downstream latency caused aggressive retries across services, multiplying requests, exhausting resources, and turning a small failure into a regional incident.

What You Will Do

- Model retry multiplication and its impact on downstream services.
- Apply timeout budgets and jitter.
- Use connection pools and circuit breakers to prevent overload.
- Validate recovery with controlled tests.

Project: Build a 3-Service Chain and Demonstrate Retry Amplification



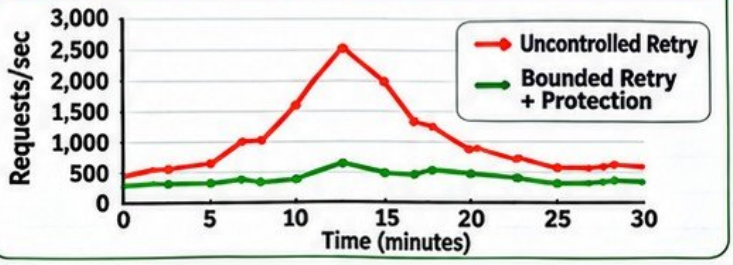
Key Mechanisms

- Timeout Budget
- Retry with Jitter
- Connection Pool
- Circuit Breaker
- Load Shedding

Retry Amplification (Example)

Scenario	User Requests	Requests to Service C	Amplification
No Retries (Control)	100	100	1×
Uncontrolled Retries (3×)	100	900	9×
Bounded Retry (2× + Jitter)	100	300	3×

System Impact: Before vs. After



Overload Protection Policy

- Set clear timeouts (e.g., 3s per call).
- Use bounded retries (e.g., max 2–3) with jitter.
- Apply circuit breakers (open on high failure rate).
- Use load shedding (reject or degrade non-critical traffic).
- Tune connection pools (limit and queue).

Deliverable

- Retry amplification measurements (before and after).
- Overload protection policy (timeouts, retry, CB, shedding).
- Recovery comparison (MTTR and system stability).

VERIQTA

The Deployment Succeeded, but the Business Lost Money

The Situation

A new release is deployed successfully and all health checks are green, but the business sees higher payment declines, duplicate orders, and increased customer latency.

What You Will Do

- Analyze the release beyond basic health checks.
- Track business metrics (payments, orders, latency, conversion).
- Implement progressive delivery (canary) with automated analysis.
- Define rollback conditions and guardrails.

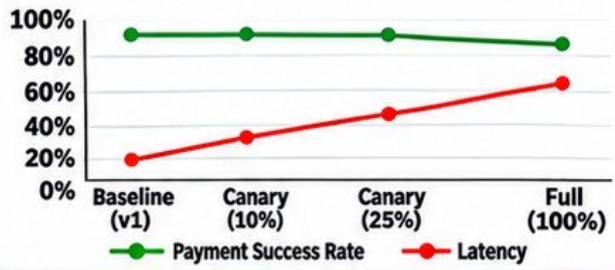
Project: Implement Progressive Delivery with Business Guardrails



Canary Comparison Dashboard (Example)

Metric	Previous (v1)	Canary (v2)	Change
Payment Success Rate	98.6%	95.1%	-3.5%
Duplicate Orders	0.02%	0.19%	+0.17%
Avg. Latency (ms)	420	780	+86%
Revenue (per hr)	$12,400	$11,100	-10.5%

Progressive Release Flow



Release Safety Policy

- Use canary or blue/green for production releases.
- Monitor business metrics (payments, orders, revenue).
- Define guardrails and automatic rollback thresholds (e.g., >2% decline in payment success or >50% latency).
- Keep a tested, one-click rollback plan.

Deliverable

- Release safety policy (with guardrails and rollback).
- Canary comparison dashboard (metrics before vs. after).
- Failed-release evidence (logs, metrics, and impact report).

The Database Is Healthy Until Every Service Reconnects at Once

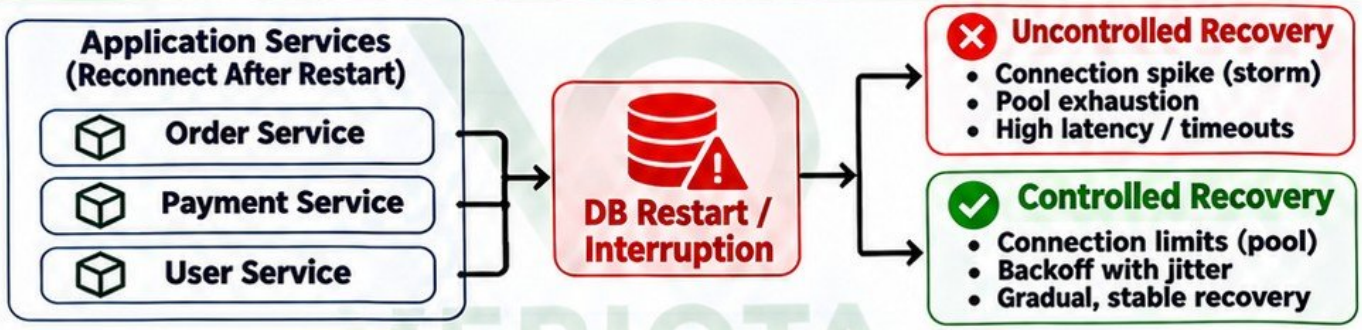
The Situation

The database appears healthy, but after a restart or failover, all services reconnect at once, causing connection storms, pool exhaustion, transaction contention, and slow recovery.

What You Will Do

- Investigate connection storms and their impact on the database.
- Analyze pool exhaustion and transaction contention.
- Simulate a database restart and observe recovery behavior.
- Implement limits, pooling, and backoff.

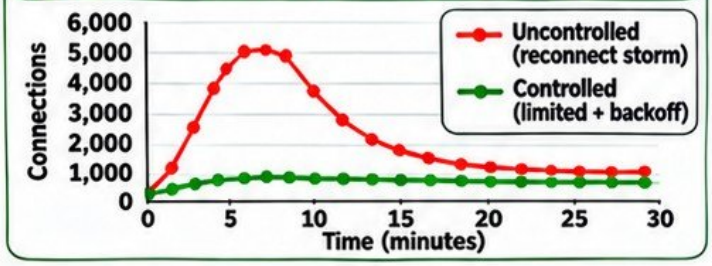
Project: Simulate a DB Interruption and Compare Recovery Approaches



Recovery Comparison (Example)

Metric	Uncontrolled	Controlled
Peak DB Connections	5,200	500
Avg. Query Latency	3,400 ms	220 ms
Error Rate	12.5%	0.8%
Time to Full Recovery	18 min	3 min

Connection Storm Pattern



Restart-Safe Configuration

- Use connection pooling (e.g., max pool size).
- Set connection limits (per service and per user).
- Implement retry with exponential backoff + jitter.
- Use readiness checks (delay traffic until DB is ready).
- Tune timeouts and transaction limits.

Deliverable

- Database recovery profile (metrics and timeline).
- Connection budget (per service and total).
- Restart-safe application configuration (pooling, backoff, timeouts, readiness).

The Runbook That Worked in Testing and Failed During the Incident

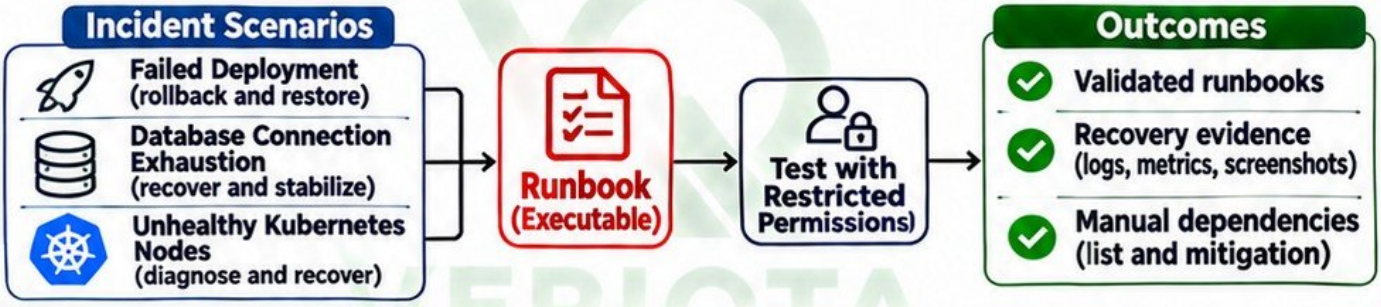
The Situation

The recovery runbook worked during testing, but failed during a real incident due to stale instructions, missing permissions, hidden assumptions, and reliance on unavailable systems.

What You Will Do

- Review and fix outdated or incorrect runbook steps.
- Create executable runbooks for key failures (deployments, DB, Kubernetes).
- Test the runbooks with restricted operator permissions.
- Identify manual steps and external dependencies.

Project: Build and Test Production-Ready Runbooks



Runbook Structure (Template)

Section	Content (Example)
Trigger	When to use (e.g., deployment failed)
Detection	How to confirm (commands, metrics)
Recovery Steps	Step-by-step commands with checks
Permissions	Required roles (e.g., read, update, delete)
Validation	How to verify recovery (service, logs, metrics)
Fallback	If the primary recovery fails
Dependencies	External systems (e.g., DB, CI/CD, DNS)

Runbook Test Results (Example)

Scenario	Test Env.	Restricted Access	Result	Time to Recover
Failed Deployment	Staging	Yes	Success	6 min
DB Conn. Exhaustion	Staging	Yes	Success	8 min
Unhealthy K8s Nodes	Staging	Yes	Success	5 min

Key Lessons

- Keep runbooks current and versioned.
- Include required permissions and access steps.
- Remove hidden assumptions (e.g., "DB is available").
- Test with realistic, restricted access and failure scenarios.

Deliverable

- Tested runbooks (3 scenarios) with clear steps.
- Recovery evidence (logs, metrics, screenshots).
- A list of remaining manual dependencies and mitigation plan.

Your First 30-Day Reliability Review: What Would You Refuse to Own?

The Situation

You are asked to take production ownership after 30 days. Before you say yes, you must review reliability, security, capacity, deployments, recovery, and operational access.

What You Will Do

- Run a production readiness review with evidence.
- Assess ownership, SLOs, security, capacity, deployments, recovery, and access.
- Classify findings: immediate blockers, accepted risks, planned improvements.
- Present your decision and recommendations to a simulated leadership panel.

Project: 30-Day Reliability Assessment and Ownership Decision

Review Areas

- Ownership & Responsibilities
- SLOs & Reliability Targets
- Security (Access, Secrets, Risk)
- Capacity & Scaling
- Deployments & Change Control
- Recovery (Backup, Restore, DR)
- Operational Access & On-Call

→ **30-Day Assessment (Evidence-Based)** →

Classify Findings

- **Immediate Blockers** (must fix before ownership)
- **Accepted Risks** (known, mitigated, and tracked)
- **Planned Improvements** (scheduled and prioritized)

Example Findings Overview

Area	Status	Key Finding (Example)
SLOs	✗	No defined SLO for checkout
Security	✗	Broad admin access (high risk)
Capacity	!	Near 80% peak utilization
Deployments	✓	Automated with rollback
Recovery	!	Backup exists, no tested restore
Access	✓	On-call and runbooks in place

Decision Framework

- **Take Ownership** (All blockers resolved, risk within tolerance)
- **Conditional Ownership** (With time-bound remediation plan)
- **Refuse Ownership** (Critical gaps that put reliability, security, or business at risk)

Deliverables

- 30-day reliability assessment (with evidence).
- Prioritized remediation backlog (issues, owner, ETA).
- A clear production ownership decision (own, conditional, or refuse) with justification.

Key Success Criteria

- Evidence-based and business-aligned review.
- Risks are clearly classified and justified.
- A defensible recommendation to the leadership panel.

VERIQTA

The Availability Zone Failed, but the Surviving Zones Could Not Handle the Traffic

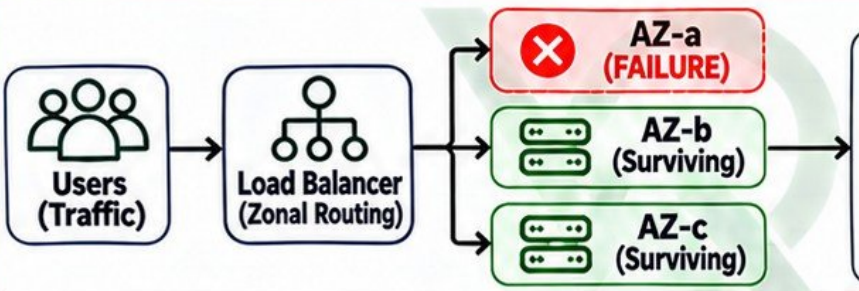
The Situation

One availability zone (AZ) fails, but the remaining zones become overloaded when all traffic is shifted to them, causing higher latency, errors, and partial service outages.

What You Will Do

- Analyze failover headroom and current capacity.
- Review zonal routing and traffic shift behavior.
- Use capacity reservations and admission control.
- Compare immediate evacuation vs. controlled degradation.

Project: Simulate an AZ Failure and Compare Traffic Handling



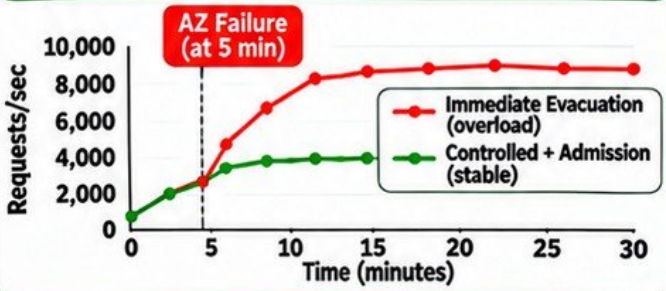
Traffic Response Options

- **Immediate Evacuation** (shift 100% to surviving AZs)
- **Controlled Degradation** (partial shift + lower capacity)
- **Admission Control** (limit new traffic, protect stability)

Capacity & Traffic Impact (Example)

Scenario	Traffic Shift	Surviving AZ Utilization	Error Rate	Avg. Latency
Baseline (All AZs)	0%	45%	0.1%	120 ms
Immediate Evacuation	100%	98%	8.4%	1,240 ms
Controlled Degradation	70%	72%	0.8%	280 ms
+ Admission Control	70% (capped)	68%	0.5%	240 ms

Traffic Shift During AZ Failure



Key Takeaways

- Always maintain failover headroom (e.g., 30–50%).
- Design zonal routing with capacity awareness.
- Use capacity reservations where needed.
- Apply admission control to protect surviving zones.

Deliverable

- AZ failure capacity model (assumptions and headroom).
- Traffic-shift thresholds (when to evacuate, degrade, or cap).
- Recovery test results (metrics, latency, errors, and stability).

Your Disaster Recovery Plan Restored Servers but Lost Customer Transactions

The Situation

A disaster recovery (DR) exercise brought the servers back online within the target time, but customers found missing orders, duplicate payments, and inconsistent transaction history.

What You Will Do

- Separate infrastructure recovery from business recovery.
- Assess RPO, RTO, and backup consistency.
- Restore the payments and orders system from backups.
- Replay a transaction log and reconcile missing and duplicate transactions.

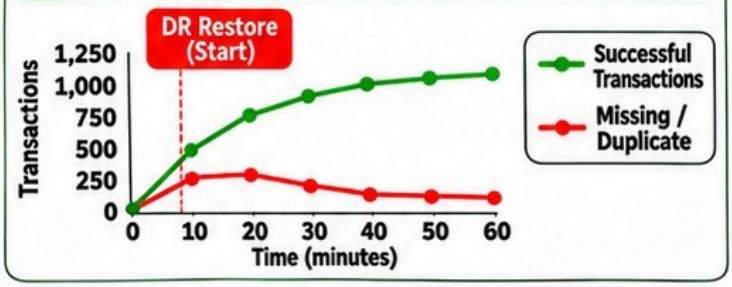
Project: Restore, Replay, and Reconcile Transactions

1. Failover / Restore (DR Site)
2. Restore from Backup (Payments & Orders)
3. Replay Transaction Log (Synthetic)
4. Reconcile Transactions (Missing & Duplicate)
5. Validate Business Recovery (Data + Orders)

Recovery Results (Example)

Metric	Target	Measured	Result
RTO (Recovery Time)	≤ 60 min	42 min	Pass
RPO (Data Loss)	≤ 5 min	8 min	Fail
Missing Transactions	0	37	Issue
Duplicate Transactions	0	12	Issue
Reconciliation Completeness	100%	100%	Pass

Transaction Flow During DR



Key Learnings

- RTO (time to restore) ≠ RPO (data you can lose).
- Use consistent, application-aware backups (e.g., database + transaction log).
- Replay and reconcile transactions after recovery.
- Make business data validation part of the DR plan.

Deliverable

- Measured recovery time (RTO: e.g., 42 minutes).
- Measured data loss (RPO: e.g., 8 minutes).
- Reconciliation report (missing: 37, duplicate: 12).
- Corrected DR design (backup, replay, validation, and duplication controls).

The Observability Platform Failed During the Outage It Was Supposed to Explain

The Situation

During a major outage, the observability platform (metrics, logs, and traces) became unavailable or incomplete, leaving the team with limited visibility and slower recovery.

What You Will Do

- Investigate telemetry loss and collector backpressure.
- Analyze high-cardinality metrics and their impact.
- Measure trace sampling and data drop rates.
- Ensure the application does not depend on monitoring availability.

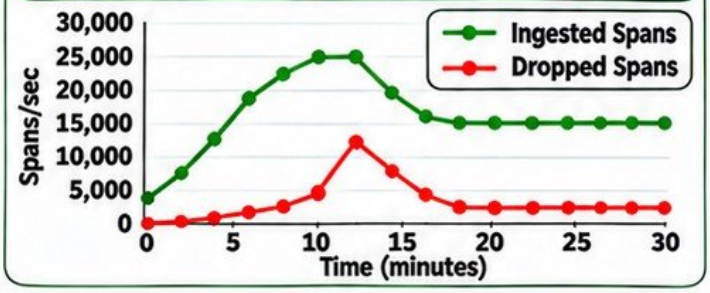
Project: Overload, Measure, and Harden the Telemetry Pipeline



Telemetry Results (Example)

Metric	Baseline	Overload (Test)	After Fix (Buffer + Sampling)
Ingest Rate (spans/sec)	5,000	25,000	15,000
Dropped Spans	0.1%	18.7%	0.9%
Metrics Delay	2 sec	120 sec	6 sec
Collector CPU	30%	95%	55%
App Overhead (p95)	8 ms	42 ms	12 ms

Spans Ingested vs. Dropped



Key Practices

- Add buffering (e.g., disk queue) for telemetry.
- Use sampling to control data volume (e.g., 10–30%).
- Enforce a cardinality budget (limit unique labels).
- Run independent health checks for the observability stack.
- Design apps to run even if monitoring is unavailable.

Deliverable

- Telemetry reliability dashboard (ingestion, drops, delay, CPU).
- Cardinality budget (e.g., ≤ 100K unique series).
- Degraded-observability response plan (what to do and how to operate).

The Queue Is Empty, but Customers Are Still Waiting

The Situation

The message queue shows zero messages, but customers are still waiting for their orders due to consumer lag, in-flight messages, failed messages, or slow business processing.

What You Will Do

- Investigate consumer lag and in-flight (unacknowledged) messages.
- Detect and handle poison (bad) messages.
- Use a dead-letter queue (DLQ) for failed events.
- Ensure correct ordering where required (e.g., per order or customer).
- Identify and fix downstream business processing bottlenecks.

Project: Build a Resilient Order-Processing Pipeline



Processing Flow (With Failures)

Event	What Happens	Result
Normal Order	Processed once	Success
Consumer Crash	Message re-delivered	Recovered
Malformed Event	Moved to DLQ	Handled
Duplicate Delivery	Ignored (idempotent)	No duplicate
Slow Business Logic	Queue delays, higher lag	Tuned / fixed

Queue & Processing Metrics (Example)

Metric	Before (Issue)	After (Fixed)
Consumer Lag	12,500 msgs	< 100 msgs
In-Flight Messages	3,200	< 50
DLQ Messages	480	0
Duplicate Orders	27	0
End-to-End Latency (P95)	18 min	2.3 min

Key Practices

- Monitor consumer lag and in-flight messages.
- Use a dead-letter queue with alerting and replay.
- Make processing idempotent (e.g., unique order_id).
- Define ordering guarantees (e.g., per customer/order).
- Set an end-to-end processing SLO (e.g., 99% ≤ 5 min).

Deliverable

- End-to-end processing SLO (e.g., 99% of orders processed ≤ 5 minutes).
- Message recovery procedure (DLQ, fix, and replay).
- Proof of duplicate-safe processing (idempotency test results and evidence).

VERIQTA

The Autoscaler Created the Incident It Was Supposed to Prevent

The Situation

The autoscaler was meant to keep the service healthy, but it triggered instability—scaling too late, too fast, and too often—causing higher latency, errors, and downstream overload.

What You Will Do

- Investigate delayed metrics and their impact on scaling decisions.
- Reproduce scaling oscillation (scale up/down repeatedly).
- Measure pod startup time and cache warming impact.
- Evaluate downstream saturation under scaling.
- Design a stable, capacity-aware scaling policy.

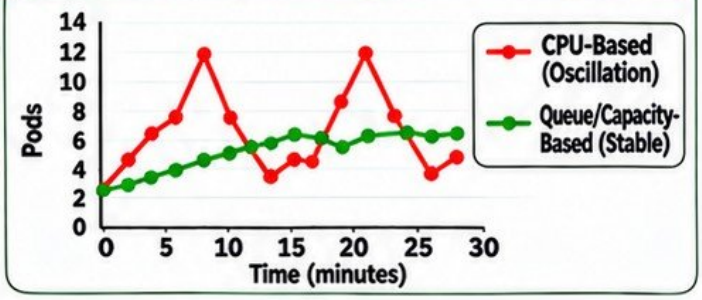
Project: Reproduce and Stabilize Autoscaling



Scaling Comparison (Example)

Approach	Avg. Latency	Scale Behavior	Result
CPU-Based (70% target)	1,240 ms	Oscillates (3 → 12 → 4)	Unstable
Queue Depth (> 1,000)	380 ms	Smooth (3 → 8)	Stable
Concurrency (> 200 req/pod)	320 ms	Controlled (3 → 6)	Better
Capacity-Aware (with warmup)	290 ms	Stable + Warmup (3 → 6)	Optimal

Scaling Behavior Over Time



Key Learnings

- Metrics delay can cause late or wrong scaling.
- Avoid scaling oscillation with proper cooldowns.
- Account for pod startup time and cache warming.
- Scale based on real workload signals (queue depth, concurrency, or throughput).
- Protect downstream services from saturation.

Deliverable

- Scaling stability report (metrics, latency, and scale events).
- Workload-specific scaling policy (e.g., HPA/KEDA with thresholds and cooldowns).
- Safe maximum capacity limits (e.g., max pods, req/sec, and downstream limits).

The Security Emergency That Became a Reliability Emergency

The Situation

A security incident (e.g., compromised credentials or expired certificate) triggered urgent changes, but the containment and rotation process disrupted critical services, causing an outage.

What You Will Do

- Investigate the impact of compromised credentials and certificate failure.
- Rotate credentials and certificates safely and with validation.
- Update IAM permissions with least privilege.
- Balance security containment with service continuity.
- Define and test a rollback path if the change fails.

Project: Simulate, Contain, Rotate, and Recover

1. Compromise (Leaked Credentials)
2. Failure (Certificate Expired / Rotation Error)
3. Contain (Revoke, Block, (Restrict Access)
4. Rotate (New Certs, New Secrets, Updated IAM)
5. Validate & Recover (Test, Restore Service, Rollback if Needed)

Incident Timeline (Example)

Step	Action	Time	Service Impact
T0	Compromise detected (credential leak)	00 min	Normal
T1	Contain (revoke & restrict)	05 min	Partial degrade
T2	Rotate (certs & secrets)	12 min	Service unstable
T3	Validate (health & access)	18 min	Service restored
T4	Post-check & monitoring	25 min	Normal

Service Health During the Event



Key Practices

- Use short-lived credentials and automate rotation.
- Rotate certificates with zero or low downtime (e.g., dual certs or blue/green).
- Apply least-privilege IAM and test permission changes.
- Validate changes before full rollout.
- Always have a tested rollback plan.

Deliverable

- Security-reliability emergency runbook (steps, owners, and decision points).
- Tested credential and certificate recovery procedure (with evidence).
- Validation and rollback results (screenshots/logs).

The Incident Commander Who Had No Authority to Stop the Damage

The Situation

A major incident was declared, but the incident commander (IC) could not take critical actions due to unclear authority, slow escalation, poor communication, and competing recovery plans.

What You Will Do

- Analyze incident roles and decision authority (IC, Ops, Comms, and stakeholders).
- Identify communication and escalation gaps.
- Evaluate conflicting recovery proposals (e.g., rollback vs. fix-forward).
- Run a realistic, time-boxed incident simulation.
- Measure response time, coordination, and outcome.

Project: Run a Timed Incident Simulation

Incident Commander (IC) (Leads, Decides, Escalates) → **Operations Lead** (Executes Fixes, Provides Technical Updates) → **Live Incident Simulation** (Conflicting Evidence + Rollback Decision) → **Communications Lead** (Stakeholder Updates, Status, Impact) → **Observers** (Evaluate Process, Decisions, and Gaps)

Example Incident Timeline (Simulation)

Time (Min)	Event	Decision / Owner
00	Incident detected (error spike)	Ops Lead
05	Incident declared & roles assigned	IC
10	Conflicting evidence (fix vs. rollback)	IC + Team
15	Escalation for approval (delayed)	IC → Leadership
20	Decision: Rollback (approved)	IC
30	Service stabilizes & monitoring recovers	Ops Lead

Response Performance (Example)

Metric	Target	Result	Status
Time to declare incident	≤ 5 min	5 min	Pass
Time to decision (rollback)	≤ 15 min	20 min	Miss
Time to recovery (service stable)	≤ 30 min	30 min	Pass
Stakeholder updates (frequency)	≤ 10 min	10 min	Pass
Overall incident duration	≤ 45 min	30 min	Pass

Key Learnings

- Define clear incident roles and decision authority.
- Establish a fast and unambiguous escalation path.
- Use a single source of truth for updates.
- Handle conflicting evidence with a structured decision process (risk, impact, and rollback criteria).
- Practice incident response regularly (simulations).

Deliverable

- Incident timeline (detection → resolution).
- Decision log (key options, evidence, owners, and final decision).
- Stakeholder updates (sample messages and timing).
- Measurable response improvements (e.g., MTTR, decision time, and communication latency).

The Postmortem Found the Cause but Changed Nothing

The Situation

Incidents kept recurring even after postmortems because the root cause was treated superficially, action items were weak, and reliability fixes never made it to the engineering backlog.

What You Will Do

- Analyze three related incidents for patterns and common contributors.
- Build a causal graph to show how failures are connected.
- Create strong, measurable action items with clear owners and due dates.
- Implement regression tests that reproduce the original failures.

Project: From Recurring Incidents to Lasting Fixes

1. **Review 3 Related Incidents** (Logs, Timelines, Postmortems)
2. **Build a Causal Graph** (Find Common Contributors)
3. **Reproduce Failures** (Regression Tests)
4. **Implement Corrective Actions** (Code, Config, Tests)
5. **Verify & Prevent** (Validate Fixes and Track Results)

Example: Three Related Incidents

Incident	Symptom	Root Cause (Actual)	Outcome
INC-101	High latency (checkout)	Not enough worker threads	Recurrence
INC-102	Request timeouts (orders)	Same thread pool saturation	Recurrence
INC-103	Partial outage (payments)	Thread saturation + no backpressure	Fixed (with tests)

Causal Graph (Example)



Key Learnings

- A good postmortem goes beyond a single "root cause."
- Look for systemic and recurring contributors (e.g., design, capacity, process, or gaps).
- Turn action items into real work (with owners, priority, and a backlog ticket).
- Add regression tests to prevent the same failure.
- Track closure and measure the impact of the fixes.

Deliverable

- Blameless postmortem (clear timeline, impact, causal graph, and learnings).
- Verified corrective actions (implemented and tested).
- Repeat-incident prevention scorecard (e.g., 0 recurrences, MTTR improvement, and closed actions).

The Automation That Saved Ten Minutes and Created a New Failure Domain

The Situation

An automated remediation was introduced to fix issues faster, but it caused repeated restarts, conflicting recovery actions, and a wider outage because it had no guardrails and no proof that it was safe.

What You Will Do

- Analyze how the automation caused a new failure domain.
- Design a safe remediation controller with guardrails and evidence.
- Implement bounded retries, dry-run, locking, audit logs, and a kill switch.
- Test that it refuses unsafe or high-risk actions.

Project: Build a Guarded Remediation Controller



Automation Control Rules (Example)

Control	Configuration (Example)	Purpose
Retries	Max = 3 (with backoff)	Prevent restart loops
Dry-Run	Enabled (default)	Simulate and validate
Locking	Distributed lock (e.g., Redis)	Avoid concurrent actions
Kill Switch	Enable/Disable (manual)	Stop automation immediately
Audit Logs	Record time, action, result	Provide evidence and traceability

Remediation Flow (With Guardrails)



Key Learnings

- Automation without guardrails can create larger incidents.
- Use dry-run, bounded retries, and distributed locks.
- Keep a kill switch and full audit logs.
- Require evidence (health checks) before and after.
- Test unsafe scenarios to ensure the system refuses risky actions.

Deliverable

- Guarded remediation workflow (architecture, code/config, and runbook).
- Safety test suite (prove it refuses unsafe actions).
- 60-day reliability improvement report (e.g., fewer repeat incidents, faster and safer recovery, with metrics and evidence).

The 30-Day SRE War Room: Prove the Platform Is Safer Than When You Inherited It

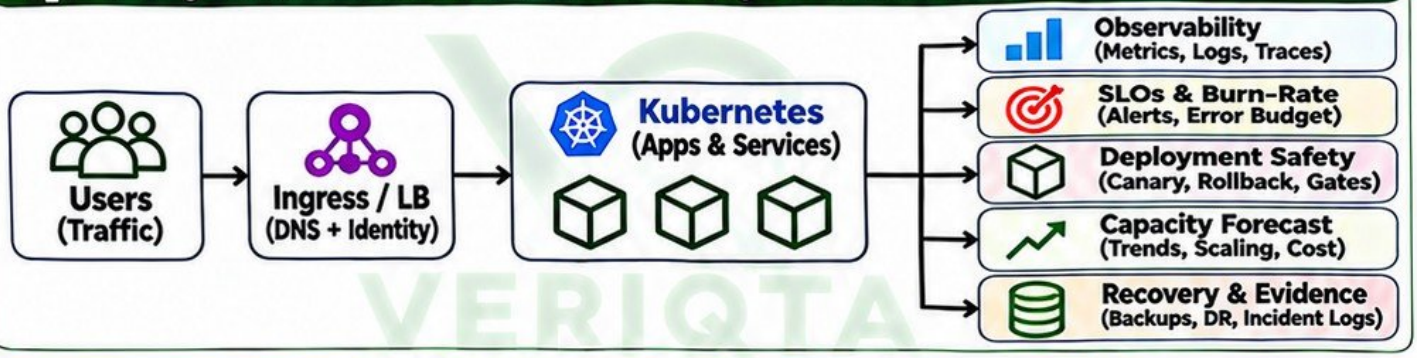
The Situation

Operate a real-world, production-like platform for 30 days through production changes, traffic spikes, security events, and infrastructure failures, and prove it is safer and more reliable than before.

What You Will Do

- Build a production reliability control plane.
- Run and respond to six major challenges.
- Maintain service SLOs and burn-rate alerting.
- Use safe deployments and dependency awareness.
- Forecast capacity and test recovery procedures.
- Collect evidence and deliver a final readiness review.

Project: Production Reliability Control Plane



The Six Major Challenges

1. A release that silently corrupts business outcomes.
2. A traffic surge that triggers autoscaling and database overload.
3. An Availability Zone failure during peak checkout activity.
4. A DNS or identity dependency failure affecting multiple services.
5. A recovery operation that risks duplicate payment processing.
6. A leadership request to reduce infrastructure cost without compromising agreed reliability objectives.

Core Capabilities

- SLOs (Targets & Error Budget)
- Burn-Rate Alerting (Fast Detection)
- Deployment Safety (Canary / Rollback)
- Dependency Awareness (Service Map)
- Capacity Forecasts (Demand & Cost)
- Tested Recovery (Backup, DR, Restore)

Success Outcome

A measurable, evidence-based improvement in reliability, safety, recoverability, and cost efficiency within 30 days.

Final Deliverables

- Working infrastructure and application code.
- Automated tests (reliability, recovery, and safety).
- Operational dashboards (SLOs, alerts, capacity, cost).
- Architecture decision records (ADRs) with trade-offs.
- Incident reports (for all six challenges).
- Cost-versus-reliability assessment (recommendations).
- Final production readiness presentation (results & evidence).